

Outline: the whole-system paper

Working title — *Proving what a second-hand computer is: an evidence-first architecture for IT asset disposition*

This replaces the earlier draft, which took one finding (the false-absence defect class) and made it the whole subject. That draft is kept intact in archive/ and becomes **one section** of this paper, §12.

What this paper is

A **systems and experience paper**: the complete design of a production ITAD platform, described layer by layer, with the engineering findings that came out of building it. 78 days, 535 commits, ~101,000 lines, one engineer.

It is not a feature list. A feature list is not publishable and is not useful to anyone six months from now. The layers are held together by one idea that the build actually followed from week one:

Every output of this system is a claim about a machine the system does not control. Its entire value is how honestly it separates what it has established from what it has assumed.

That thesis is what makes the offline queue, the boot stick, the registry reader, the wipe ladder, the certificate and the Autopilot limit *one system* rather than seven projects.

Structure

Part I — The problem

§	Section	What it establishes
1	Introduction	What ITAD must prove, to whom, and what happens when it is wrong
2	Requirements and constraints	Warehouse floor, no network, non-expert operator, legally consequential output
3	System overview	The four deployables, the data flow, the trust boundary

Part II — The layers

§	Section	The layer
4	The inventory model	Lot → Asset → Hardware; the three stock tiers; ownership; the activity log
5	Offline-first capture	PowerSync, the durable queue, scanning; why the dangerous failure is the quiet one
6	The audit station	Booting a machine that is not yours; the boot chain; kiosk hardening; the ship boundary
7	Deep inspection	Offline registry hives, device locks, management state, drive health, OS capture
8	Erasure and certification	The method ladder, per-drive verdicts, hidden areas, read-back, signed certificates
9	Functional testing	The technician-confirmed tests, and the three-valued verdict
10	The back office	Reports and BI, the operational dashboard, fail-closed permissions, accessibility

Part III — What the building taught

§	Section	The finding
11	The knowability framework	Four tiers of what a machine can be made to tell you; the hard limit at Autopilot
12	Evidence integrity	The false-absence class: 32 call sites, and the recurrence result
13	The discipline of deletion	Three working modules removed, and why that is why the system stayed buildable
14	Engineering practice	Design records, the test strategy, migration ordering, the two-push deploy

Part IV — Closing

§	Section
15	Results: what the system does, and at what scale
16	Limitations and threats to validity
17	Lessons for anyone building an evidence-producing system
18	Related work
19	Conclusion

Length and venue

Roughly **12,000–15,000 words**. That is a systems paper, not a magazine article, and the earlier venue arithmetic no longer applies: IEEE Software was never going to hold this and is dropped.

Realistic homes: **arXiv cs.SE** (no limit, citable, recommended first), **ICSE SEIP** or **ESEM industry track** if a refereed venue is wanted, or a journal such as *IEEE Software's* longer-form siblings. Decide after the draft exists, not before.

Rules for this draft

1. **Every layer gets the same treatment**: what it does, why it is shaped that way, what was rejected, and what is still unproven.
2. **No figure is written before it is measured**. Three were wrong in the previous draft when finally checked. Every number here is recomputed from the repository or the dataset before the section carrying it is committed.
3. **Unproven is said out loud**. Anything that has not run on real hardware is named as such, in the section that claims it.
4. **The failures stay in**. A paper describing only what worked is a brochure.